

Автоматизация тестирования Node.js REST API с использованием CI

GitHub Actions: от git push до автоматического запуска Jest и Supertest

Цели лекции

- Понять назначение Continuous Integration в backend-разработке.
- Разобрать базовые понятия GitHub Actions: workflow, event, job, runner, step, action.
- Научиться автоматически устанавливать зависимости и запускать тесты Node.js-приложения.
- Подключить генерацию Prisma Client и тестовую PostgreSQL для integration tests.
- Научиться читать результат CI и находить причину failed workflow.

Главная идея

Главная идея: CI автоматически повторяет проверку проекта после изменений кода.

Что будем автоматизировать

● Получение кода

GitHub Actions загружает текущую версию репозитория.

● Зависимости

npm ci устанавливает зависимости строго по package-lock.json.

● Тесты

Jest и Supertest проверяют бизнес-логику и REST API.

● Важно

CI не пишет тесты за разработчика — он автоматически запускает уже созданные тесты.

● Node.js

Runner получает нужную версию Node.js — в нашем курсе Node.js 22.

● Prisma

prisma generate создаёт Prisma Client.

● Результат

Workflow завершается статусом success или failure.



1. Что такое Continuous Integration

Зачем backend-проекту автоматическая проверка после изменений

Ручная проверка и CI

Только вручную

- Разработчик должен помнить все команды.
- Тесты могут быть случайно пропущены.
- Результат зависит от локального окружения.
- Ошибка может попасть в main.

С Continuous Integration

- Проверки запускаются автоматически.
- Одинаковые шаги выполняются для каждого изменения.
- Результат сохраняется в GitHub.
- Failure виден до дальнейшего развёртывания.

Определение CI

- Continuous Integration (CI) — практика, при которой изменения кода регулярно интегрируются в общий репозиторий и автоматически проверяются сборкой, тестами и другими заданными проверками.
- CI запускается по событию: push, pull request или вручную.
- Команды выполняются в отдельном временном окружении.
- Каждый запуск имеет журнал выполнения.
- Успех или ошибка фиксируются как статус workflow.

Главная идея

В этой лекции CI =
автоматическая проверка
Node.js REST API, а не
автоматический deployment.

CI и CD — не одно и то же

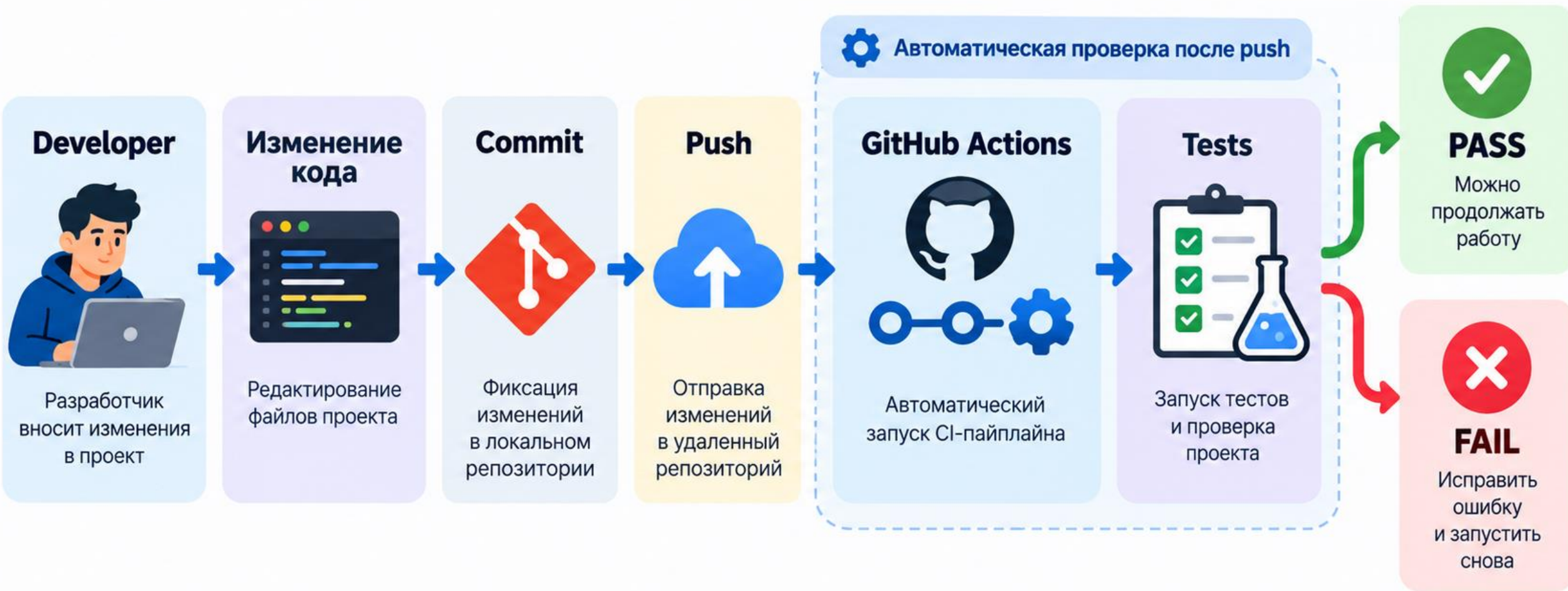
CI

- Получить код.
- Установить зависимости.
- Сгенерировать Prisma Client.
- Запустить unit tests.
- Запустить integration tests.

CD

- Подготовить release.
- Развернуть приложение.
- Обновить production.
- Выполнить post-deploy проверки.
- При необходимости выполнить rollback.

Путь изменения кода через CI



Что CI даёт проекту Internet Shop

● Раннее обнаружение ошибок

Ошибка обнаруживается сразу после отправки изменений.

● Повторяемость

Для каждого commit выполняется одинаковая последовательность команд.

● Контроль main

Перед объединением изменений можно видеть результат тестов.

● История

GitHub хранит историю запусков и logs.

● Когда CI особенно полезен

CI особенно полезен, когда проект состоит из
Router → Controller → Service → Repository → PostgreSQL.

Что будет делать наш CI

- 1 Checkout** Получить исходный код Internet Shop из GitHub.
- 2 Setup Node.js** Подготовить Node.js 22 на GitHub runner.
- 3 npm ci** Установить зависимости по package-lock.json.
- 4 Prisma generate** Сгенерировать Prisma Client.
- 5 Database preparation** Подготовить тестовую PostgreSQL и применить migrations.
- 6 npm test** Запустить Jest и Supertest и получить PASS или FAIL.



2. Основы GitHub Actions

Workflow, event, job, runner, step и action

Основные понятия GitHub Actions

- Workflow — автоматизированный процесс, описанный YAML-файлом.
- Event — событие, которое запускает workflow.
- Job — отдельное задание внутри workflow.
- Runner — машина, на которой выполняется job.
- Step — последовательный шаг внутри job.
- Action — готовое повторно используемое действие, например checkout или setup-node.

Структура GitHub Actions



Где хранится workflow

TEXT

```
internet-shop/  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
├── src/  
├── tests/  
├── prisma/  
├── package.json  
└── package-lock.json
```

Когда запускать workflow

YAML

```
name: CI
```

```
on:
```

```
  push:
```

```
    branches: [ main ]
```

```
  pull_request:
```

```
    branches: [ main ]
```

Что важно

- name задаёт понятное имя workflow в GitHub Actions.
- push запускает CI после отправки изменений в ветку main.
- pull_request запускает CI при проверке изменений перед слиянием с main.
- Один workflow может реагировать сразу на несколько событий.

GitHub-hosted runner

- Runner — вычислительное окружение, в котором GitHub Actions выполняет команды workflow.
- Для учебного CI используем runs-on: ubuntu-latest.
- Runner создаётся для запуска job и не является вашим локальным ПК.
- Поэтому зависимости и Prisma Client необходимо подготовить заново.
- Файлы локального .env автоматически на runner не передаются.

Главная идея

Workflow должен содержать всё необходимое для воспроизводимой проверки проекта.

Job и steps

YAML

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v7

      - name: Show Node version
        run: node --version
```

Что важно

- jobs содержит задания workflow.
- test — имя нашего job.
- runs-on выбирает runner.
- steps выполняются сверху вниз.
- uses запускает готовое action.
- run выполняет команду shell.

Подготовка Node.js 22

YAML

```
- name: Setup Node.js
  uses: actions/setup-node@v7
  with:
    node-version: '22'
    cache: 'npm'
```

Что важно

- setup-node устанавливает выбранную версию Node.js.
- node-version: 22 соответствует окружению нашего курса.
- cache: npm позволяет повторно использовать npm cache между запусками.
- Кэш не заменяет npm ci: зависимости всё равно устанавливаются в рабочий каталог.



3. Первый CI workflow для Node.js

От YAML-файла до первого зелёного запуска

Минимальный workflow

YAML

```
name: Node.js CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: '22'
          cache: 'npm'
```

Как выполняется минимальный workflow

YAML

steps:

- uses: actions/checkout@v7
- uses: actions/setup-node@v7
 - with:
 - node-version: '22'
 - cache: 'npm'
- run: npm ci
- run: npm test

Что важно

- Сначала runner получает содержимое репозитория.
- Затем настраивается Node.js 22.
- npm ci устанавливает зависимости.
- npm test запускает тестовый script из package.json.
- Если команда возвращает ненулевой exit code, step и job получают failure.

npm ci и npm install в CI

npm ci

- Ориентирован на автоматизированные окружения.
- Требуется package-lock.json.
- Устанавливает версии из lockfile.
- Даёт более воспроизводимый результат.

npm install

- Обычная установка зависимостей для разработки.
- Может изменять lockfile.
- Уместен при добавлении и обновлении packages.
- Не является основным выбором для нашего CI.

Как CI находит тесты

JSON

```
{  
  "scripts": {  
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand"  
  }  
}
```

Что важно

- GitHub Actions не знает, как именно тестировать проект.
- Workflow вызывает `npm test`.
- `npm` читает `script test` из `package.json`.
- Jest запускает тестовые файлы проекта.
- `--runInBand` выполняет тесты последовательно, что удобно для учебной `test database`.

Локальный запуск и GitHub runner

Локальная машина

Среда разработчика



```
$ npm ci
$ npm test
```

Одинаковые
команды



Те же команды
выполняются
в обеих средах

GitHub runner

Облачная среда



```
$ npm ci
$ npm test
```



Разные среды, но одинаковый шаг установки зависимостей и запуска тестов.

Что проверить до отправки workflow

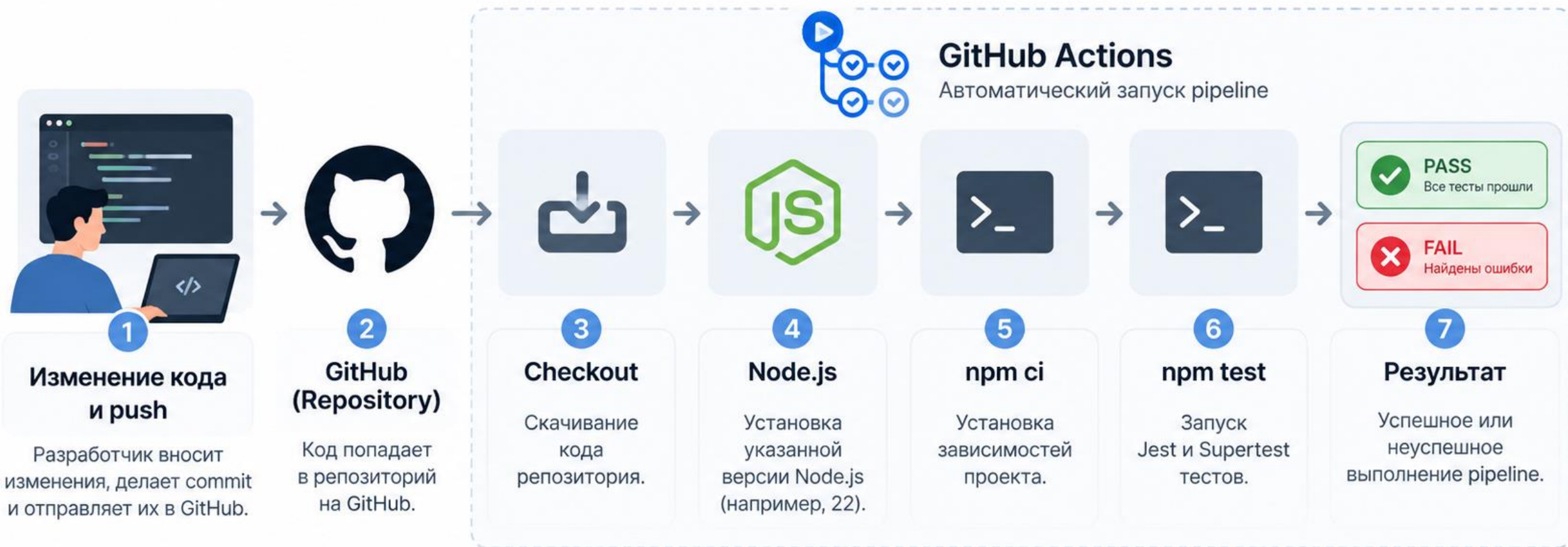
- ✓ package.json содержит рабочий script test.
- ✓ package-lock.json добавлен в Git.
- ✓ Все тестовые файлы сохранены в репозитории.
- ✓ Тесты локально проходят командой npm test.
- ✓ В проект не добавлен настоящий .env с паролями.
- ✓ Путь .github/workflows/ci.yml написан без ошибок.

Первый запуск GitHub Actions

- 1 Создать ci.yml** Добавить workflow в .github/workflows.
- 2 Commit** Зафиксировать workflow вместе с изменениями проекта.
- 3 Push** Отправить commit в GitHub.
- 4 Открыть Actions** Перейти во вкладку Actions репозитория.
- 5 Открыть job** Просмотреть steps, logs и итоговый status.

Минимальный CI pipeline

Автоматический запуск тестов при каждом push в репозиторий



Каждое изменение кода автоматически проверяется тестами.

4. Автоматический запуск Jest и Supertest

Используем в CI тесты, которые уже разработаны на предыдущих занятиях

Какие тесты уже есть в проекте

Unit tests — Jest

- Проверяют Service layer.
- Используют expect и matchers.
- Зависимости могут заменяться jest.fn().
- Работают быстро и локально.

Integration tests — Supertest

- Отправляют HTTP-запросы в Express app.
- Проверяют status code и response body.
- Проверяют middleware и маршрутизацию.
- Некоторым тестам нужна test database.

Один npm script может запускать весь набор тестов

JSON

```
{
  "scripts": {
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand",
    "test:watch": "node --experimental-vm-modules node_modules/jest/bin/jest.js --watch"
  }
}
```

Важный принцип CI

- Команды проверки в CI должны максимально совпадать с командами, которые разработчик может выполнить локально.
- Локально: `npm test`.
- В GitHub Actions: `npm test`.
- Не создаём отдельную скрытую логику только для CI.
- Если CI падает, разработчик должен иметь возможность воспроизвести проблему локально.

Главная идея

**Один проект — один
понятный способ запуска
тестов.**

Что происходит при failed test

TEXT

```
FAIL tests/products.test.js
```

```
Expected: 200
```

```
Received: 500
```

```
Tests: 1 failed, 12 passed
```

```
Process completed with exit code 1.
```

Что важно

- Jest сообщает, какой test завершился ошибкой.
- В logs остаются expected и received значения.
- npm test возвращает exit code 1.
- GitHub Actions помечает step, job и workflow как failed.

Требования к тестам для CI

- ✓ Тесты не должны требовать ручного ввода.
- ✓ Тесты не должны зависеть от файлов, которых нет в репозитории.
- ✓ Не использовать production database.
- ✓ Тестовые данные должны быть воспроизводимыми.
- ✓ После тестов не должно оставаться состояния, влияющего на следующий тест.
- ✓ Секреты не должны быть записаны прямо в test files.

Environment variables в CI

- Локальный `.env` обычно находится в `.gitignore` и не попадает в GitHub.
- Неопасные тестовые значения можно задать через `env` в workflow.
- Чувствительные значения следует хранить в GitHub Secrets.
- `NODE_ENV=test` помогает приложению выбрать тестовое поведение.

Главная идея

Нельзя исправлять CI добавлением настоящего `production .env` в репозиторий.

env и secrets

Обычный env

- Тестовое имя базы данных.
- `NODE_ENV=test`.
- Неопасные параметры.
- Задаются прямо в workflow.

GitHub Secrets

- Пароли внешних сервисов.
- API tokens.
- Закрытые connection strings.
- Не записываются открытым текстом в YAML.

Тестовые переменные workflow

YAML

```
jobs:
  test:
    runs-on: ubuntu-latest
    env:
      NODE_ENV: test
      JWT_SECRET: ci-test-secret

    steps:
      - uses: actions/checkout@v7
      - run: npm test
```

Что важно

- env на уровне job доступен всем steps этого job.
- NODE_ENV сообщает приложению, что выполняются тесты.
- Для учебного CI допустимо использовать искусственный JWT_SECRET.
- Production secret сюда помещать нельзя.

CI как контрольная точка

Автоматическая проверка качества кода при каждом изменении



Tests passed

Все тесты прошли успешно



Push

в репозиторий



GitHub Actions

Запуск workflow



Checkout code



Setup Node.js



npm ci



Run tests



GitHub Actions

Success

✓ Checkout code	2s
✓ Setup Node.js (v22)	3s
✓ npm ci	12s
✓ Run unit tests (Jest)	8s
✓ Run integration tests (Supertest)	15s

Tests: **56 passed, 0 failed**

Time: 12.345s

🌟 Workflow completed successfully!



Код соответствует требованиям,

можно переходить к следующим этапам разработки.



Tests failed

Найдены ошибки, workflow остановлен



Push

в репозиторий



GitHub Actions

Запуск workflow



Checkout code



Setup Node.js



npm ci



Run tests

Ошибка в тестах



GitHub Actions

Failed

✓ Checkout code	2s
✓ Setup Node.js (v22)	3s
✓ npm ci	11s
✗ Run unit tests (Jest)	6s

FAIL tests/user.test.js

- should return user by id
Expected 200 to be 404
+ Expected
- Received
- 200
+ 404

Tests: **1 failed, 55 passed**

Time: 7.231s

Error: Process completed with exit code 1.



Workflow остановлен.

Исправьте ошибки, сделайте новый commit и push.



CI — это автоматическая контрольная точка,

которая не позволяет двигаться дальше, если тесты не прошли.

5. Prisma и PostgreSQL в CI

Как подготовить Prisma Client и временную test database для integration tests

Почему в CI нужен prisma generate

- Runner начинает работу с чистого окружения.
- После `npm ci` необходимо подготовить сгенерированный Prisma Client.
- Команда `npm run prisma generate` выполняется до тестов.
- `DATABASE_URL` должен быть доступен, если конфигурация Prisma обращается к нему при загрузке.

Главная идея

CI должен повторять технические шаги, необходимые приложению после чистой установки.

Добавляем Prisma Client в pipeline

YAML

- name: Install dependencies
run: npm ci
- name: Generate Prisma Client
run: npx prisma generate
- name: Run tests
run: npm test

Что важно

- Сначала устанавливаются npm dependencies.
- Затем генерируется Prisma Client.
- Только после подготовки приложения запускаются тесты.

Integration tests и база данных

- Unit tests обычно не требуют реальной PostgreSQL.
- Integration tests могут обращаться к Repository/Prisma.
- Для CI нужна отдельная test database.
- Production database использовать для тестов нельзя.
- База должна быть доступна runner только на время проверки.

Главная идея

Test database должна быть изолирована от development и production.

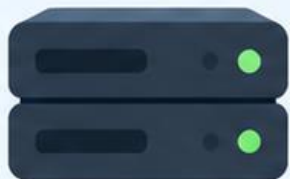
GitHub Actions + временная PostgreSQL

Автоматическое тестирование API с использованием реальной базы данных



GitHub Actions Runner

Виртуальная машина в облаке GitHub



- 1 Запуск workflow
- 2 Установка Node.js
- 3 Установка зависимостей
- 4 Запуск миграций
- 5 Запуск тестов



Prisma

Работа с базой данных



Подключение к базе данных



prisma generate

Генерация клиента



prisma migrate deploy

Применение миграций



Prisma Client

Доступ к данным в тестах



Test PostgreSQL

Временная база данных в контейнере



PostgreSQL

(service в GitHub Actions)

Порт: 5432
База: testdb
Пользователь: postgres
Пароль: postgres

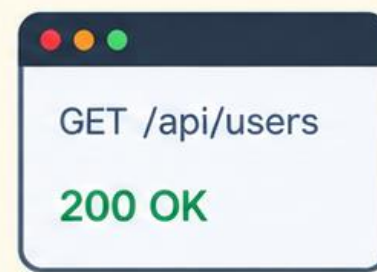


Создаётся автоматически для каждого запуска и удаляется после завершения.



Supertest

Тестирование API



Отправка HTTP-запросов к API (через Supertest)



Проверка ответов



Проверка данных в PostgreSQL



Автоматический запуск в CI



Результат: автоматически проверяем API с реальной базой данных, без ручной настройки и установки PostgreSQL на своём компьютере.



Каждый запуск — чистое окружение

Временная PostgreSQL service для job

YAML

```
services:
  postgres:
    image: postgres:17
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shop_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd="pg_isready -U postgres -d shop_test"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=5
```

Что важно

- GitHub Actions создаёт временную PostgreSQL для job.
- POSTGRES_DB задаёт отдельную базу shop_test.
- Port 5432 доступен runner через localhost.
- Health check позволяет дождаться готовности PostgreSQL.
- После завершения job временная служба удаляется автоматически.

DATABASE_URL для test database

YAML

env:

NODE_ENV: test

JWT_SECRET: ci-test-secret

DATABASE_URL: postgresql://postgres:postgres@localhost:5432/shop_test?schema=public

Что важно

- localhost используется потому, что job выполняется непосредственно на Ubuntu runner.
- 5432 опубликован временной PostgreSQL service.
- shop_test — тестовая база, созданная специально для CI.
- Эта строка не указывает на production database.

Подготавливаем схему тестовой базы

YAML

- name: Generate Prisma Client
run: npx prisma generate
- name: Apply test database migrations
run: npx prisma migrate deploy
- name: Run Jest and Supertest
run: npm test

Что важно

- prisma generate создаёт клиент.
- prisma migrate deploy применяет уже существующие migrations из репозитория.
- После этого структура test database соответствует проекту.
- npm test запускается только после подготовки схемы.

Полный учебный workflow ci.yml (ч.1)

YAML

```
name: Internet Shop CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    env:
      NODE_ENV: test
      JWT_SECRET: ci-test-secret
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/shop_test?schema=public
```

Полный учебный workflow ci.yml (ч.2)

YAML

```
services:
  postgres:
    image: postgres:17
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shop_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd="pg_isready -U postgres -d shop_test"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=5
```

Полный учебный workflow ci.yml (ч.3)

YAML

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v7

  - name: Setup Node.js
    uses: actions/setup-node@v7
    with:
      node-version: '22'
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Generate Prisma Client
    run: npx prisma generate

  - name: Apply database migrations
    run: npx prisma migrate deploy

  - name: Run tests
    run: npm test
```

Как читать полный workflow

- 1 Trigger** `push` или `pull request` для `main` запускает workflow.
- 2 Runner** GitHub создаёт Ubuntu runner.
- 3 PostgreSQL** Создаётся временная test database.
- 4 Node.js** `Checkout` и `setup-node` подготавливают проект и Node.js 22.
- 5 Application preparation** `npm ci`, `prisma generate` и `prisma migrate deploy` готовят приложение.
- 6 Tests** `npm test` запускает Jest и Supertest и определяет итоговый status.



6. Результаты CI и диагностика ошибок

Как понять, почему workflow завершился failure



Как устроен результат GitHub Actions

От запуска workflow до конкретной причины ошибки



GitHub Actions

1 Workflow

Запуск всего процесса

.github/workflows
test.yml

```
name: Tests
on:
  push:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm test
```

Workflow — это файл с описанием всего процесса, который запускается при событии (например, push).

2 Job

Набор шагов, выполняемый на одном runner

Jobs

✖ test	28s
○ lint	12s
○ build	18s

Job — это группа шагов (steps), которая выполняется на одном виртуальном раннере (runner). В одном workflow может быть несколько jobs.

3 Step

Отдельный шаг внутри job

✖ test	failed 28s
✔ Set up job	2s
✔ Checkout code	3s
✔ Setup Node.js	4s
✔ Install dependencies	8s
✖ Run tests	6s
○ Post job	1s

Step — это конкретная команда или действие (например, запуск npm test). Каждый шаг имеет свой статус: success, failed, skip.

4 Logs

Подробный вывод шага

```
> ✖ Run tests
1 > npm test
2
3 > api@1.0.0 test
4 > jest
5
6 FAIL tests/user.test.js
7   ● GET /api/users
8     expect(received).toBe(200)
9
10    Expected: 200
11    Received: 404
12
13    at tests/user.test.js:12:28
14
15 Test Suites: 1 failed, 1 total
16 Tests:      1 failed, 3 passed
17 Time:       2.345 s
18 Ran all test suites.
```

Logs — это полный вывод команды. Здесь видно, что именно произошло и на какой строке возникла ошибка.

5 Конкретная причина ошибки

Анализируем и исправляем

```
FAIL tests/user.test.js
  ● GET /api/users
    expect(received).toBe(200)
    Expected: 200
    Received: 404
    at tests/user.test.js:12:28
```

Как найти и исправить:

- 1 Смотрим сообщение об ошибке
- 2 Переходим к указанному файлу и строке
- 3 Исправляем код или тест
- 4 Делаем новый commit и push



Результат GitHub Actions — это не просто зелёный или красный статус,

а подробная информация на каждом уровне: workflow → job → step → logs → конкретная причина ошибки.



Чем раньше ошибка найдена, тем дешевле её исправить.

Типичные причины failure

Симптом	Вероятная причина	Что проверить
npm ci failed	Нет или неактуален package-lock.json	package.json и package-lock.json должны соответствовать друг другу
prisma generate failed	Ошибка Prisma config или environment	DATABASE_URL и prisma.config.ts
Database connection refused	PostgreSQL ещё не готова или неверный URL	service health check, localhost, port 5432
Migration failed	Проблема migration files	prisma/migrations и состояние test database
Jest failed	Ошибка кода или теста	Первый failed test и stack trace
Supertest получает 500	Ошибка API во время запроса	server logs, error middleware, test data

Алгоритм диагностики failed workflow

- 1 Найти красный step** Не начинать анализ со всего workflow сразу.
- 2 Открыть logs** Найти первое содержательное сообщение об ошибке.
- 3 Определить слой** Dependencies, Prisma, database, Jest или API.
- 4 Воспроизвести локально** Запустить ту же команду на локальном проекте.
- 5 Исправить причину** Не маскировать ошибку отключением test.
- 6 Commit и push** Новый push запускает CI повторно.

Практическая мини-задача

Workflow завершился failure на шаге Run tests: один Supertest-тест ожидал 200, но получил 401.

Задание

- Определите, на каком уровне находится проблема: установка зависимостей, Prisma, database или HTTP API.
- Проверьте, требуется ли Authorization header для endpoint.
- Сравните локальный запуск того же теста.
- Исправьте test setup или код приложения в зависимости от причины.

Критерии проверки

- Студент не перезапускает workflow наугад.
- Находит конкретный failed step.
- Использует logs и воспроизводит ошибку локально.
- После исправления новый CI run получает success.

Проверка понимания

- 1 Что запускает workflow: event, runner или package-lock.json?
- 2 Почему в CI для Node.js-проекта предпочтителен npm ci?
- 3 Что произойдёт с workflow, если npm test завершится с exit code 1?
- 4 Зачем выполнять prisma generate на чистом runner?
- 5 Почему integration tests нельзя запускать против production database?

Диагностика CI

От ошибки
к работающему
pipeline

Работаем
→ Стабильный CI
→ Быстрые релизы
→ Уверенный код!



Упал workflow



Найди шаг
с ошибкой

```
....  
> npm test  
✗ Test failed  
  
Error: Cannot find module  
  at Object.<anonymous>  
  at runTest (test.js:10)  
....
```

Изучи logs

\$ npm test



Повтори локально



Исправь причину



git push

Сделай новый push



Идея: CI показывает, где упало, а разработчик воспроизводит проблему локально, исправляет её и повторно запускает pipeline новым push.



Меньше ошибок
Больше фич!

Итоги лекции

1

CI автоматически проверяет изменения проекта и не заменяет написание тестов.

2

GitHub Actions описывается YAML-файлами в `.github/workflows`.

3

Базовый Node.js CI: `checkout` → `setup-node` → `npm ci` → `npm test`.

4

Для проекта с Prisma выполняем `prisma generate` перед тестами.

5

Integration tests должны использовать отдельную test database.

6

GitHub Actions может подготовить временную PostgreSQL на время job.

7

Failure анализируем по конкретному step и его logs.

8

Следующая тема: способы развёртывания Node.js REST API и публикация на PaaS Render.